TD02 - C Exercises

Exercises from CM-all-2025

Louis Ledoux

2026-07-05

Table of contents

1 C Exercises

Live pacing: about 8 min

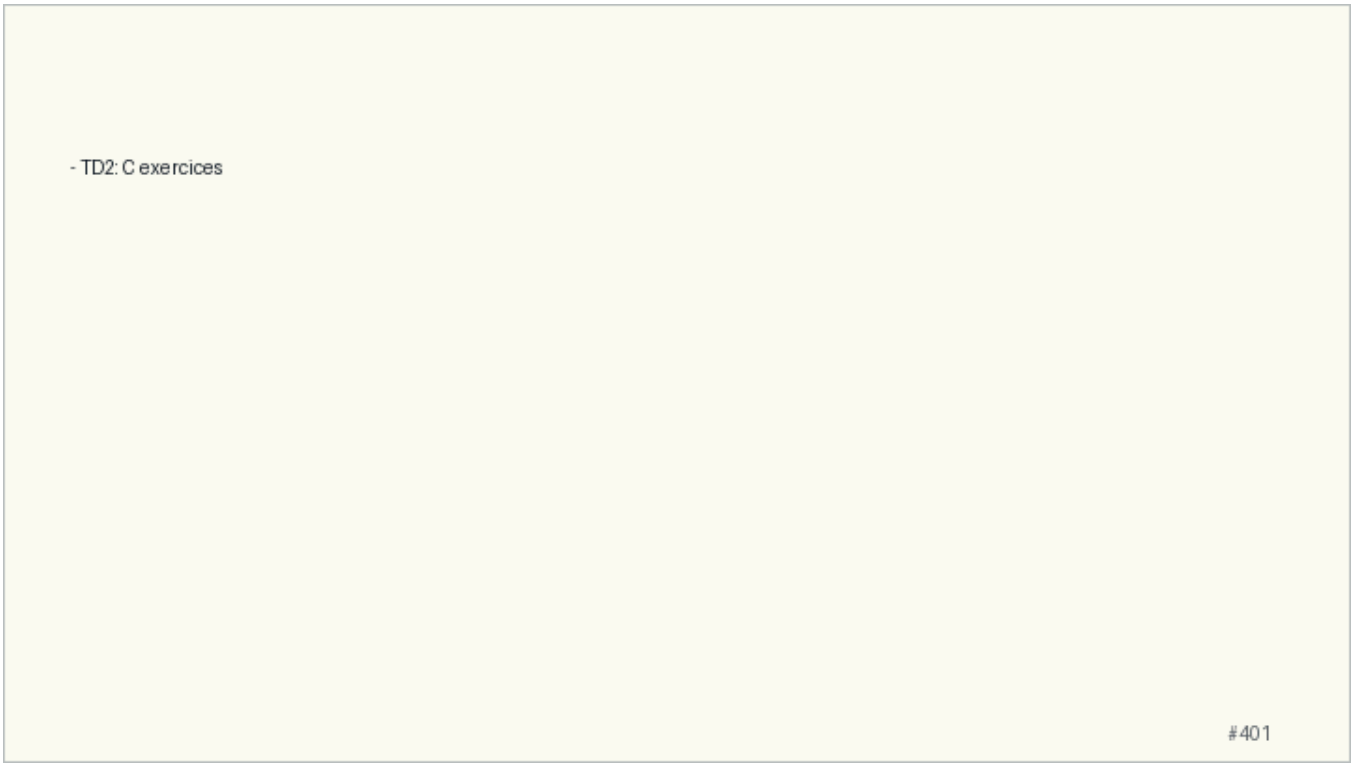
Question. Can students transfer the lecture model to unfamiliar code?

Core claim. Exercises should force a trace: values, addresses, calls, files, or ownership.

Target capability. Students can justify an answer with a state diagram, not only a final value.

1.1 Main Path

- Start from the question: Can students transfer the lecture model to unfamiliar code?
- Build the model: Exercises should force a trace: values, addresses, calls, files, or ownership.
- End with a checkable action: Students can justify an answer with a state diagram, not only a final value.
- Keep only this slide on the horizontal path during the live lecture.



1.2 Worked Example

- Use this as the live trace: Ask for a prediction first, then require the memory or process trace that supports it.
- Representative source slide: 402
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

1.3 Anecdote Or Motivation

The best exercise answers include a small drawing; C rewards explicit state.

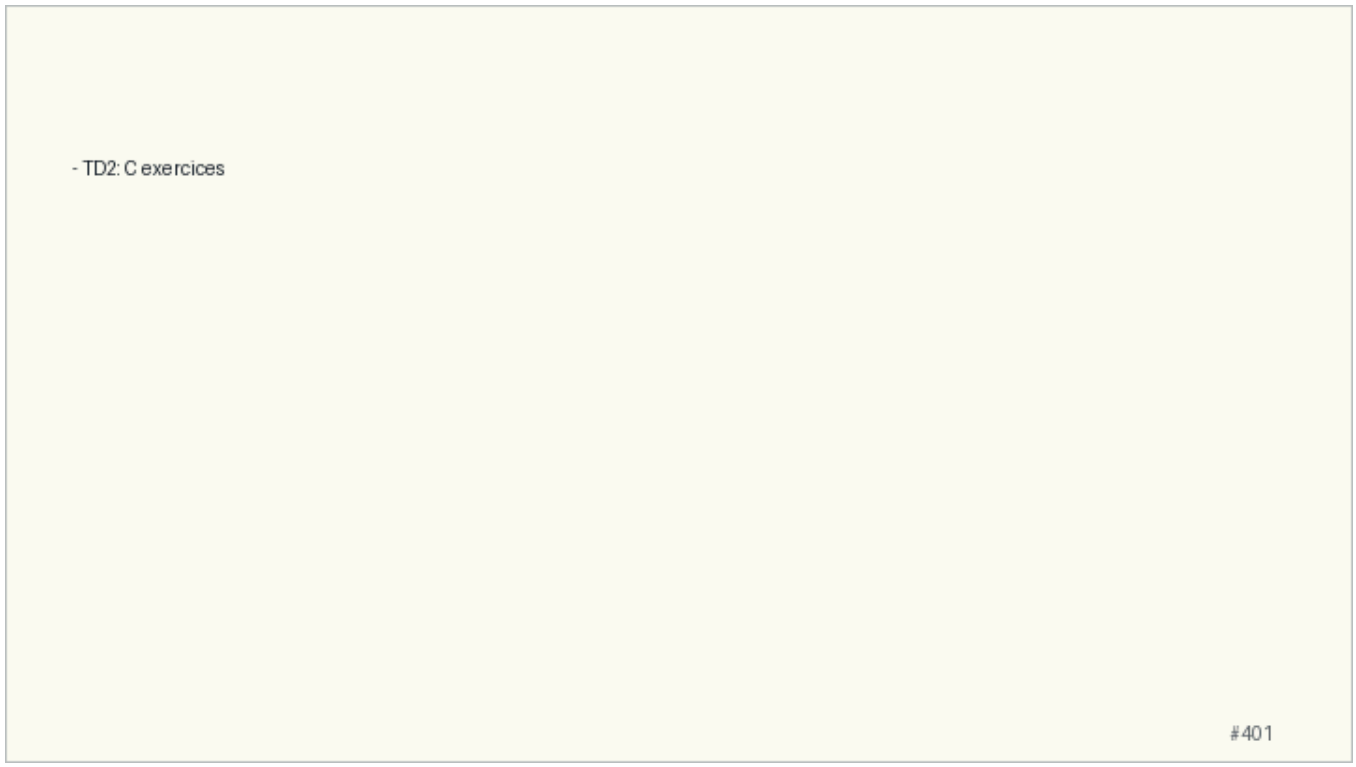
- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

1.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

1.5 Backup Index

Original slides 401-410



slide 401

- Exercice 1: Basic I/O

- What prints the following program? (assume array nom is stored at the address 0x3472A000)

```
#include<stdio.h>
int main(){
char nom[20] = "Balthazar";
double solde = 1000.25;
int age = 21;
printf("Hello %s, you have %d years\n", nom, age);
printf("Your age is written %x in hex\n", age);
printf("Your name start with the letter %c\n", nom[0]);
printf("Your bank account balance is %lf\n", solde);
printf("The address of the table name is %p\n", nom);
}
```

#402

slide 402

Exercice 2: Pointers

- Assume the following instructions int x=0; int y=0; int* p=NULL; p = &x; x=2; y=4; *p = y;
x = 5; y = *p; p = &y;
- Provide the values of x, y and p

#403

slide 403

Exercice 3: Pointers

- Give the output of the program

```
#include <stdio.h>
void echange(int * a, int b){
int sauve; sauve = *a; *a = b; b = sauve;
}
void echange2(int * a, int * * b){
int sauve; sauve = *a; *a = * *b; * *b = sauve;
}
void echange3(int * a,int * b){
int sauve; sauve = *a; *a = *b; *b = sauve;
}
int main(){
int a, b, d; int * c = &d; a = 1; b = 2; (*c) = 3;
echange(&a,b); printf("a= %d, b= %d\n",a,b);
echange2(&a,&c); printf("a= %d, c= %d\n",a,*c);
echange3(&a,&b); printf("a= %d, b= %d\n",a,b);
echange3(&b,c); printf("b= %d, c= %d\n",b,*c);
echange(&a,&b); printf("a= %d, b= %d\n",a,b);
return 0;
}
```

#404

slide 404

Exercice 4: Strings

- Write a function called reverse that reverses the characters of a string, without declaring a second array. The prototype of the function is the following:
- int reverse(char * str);

#405

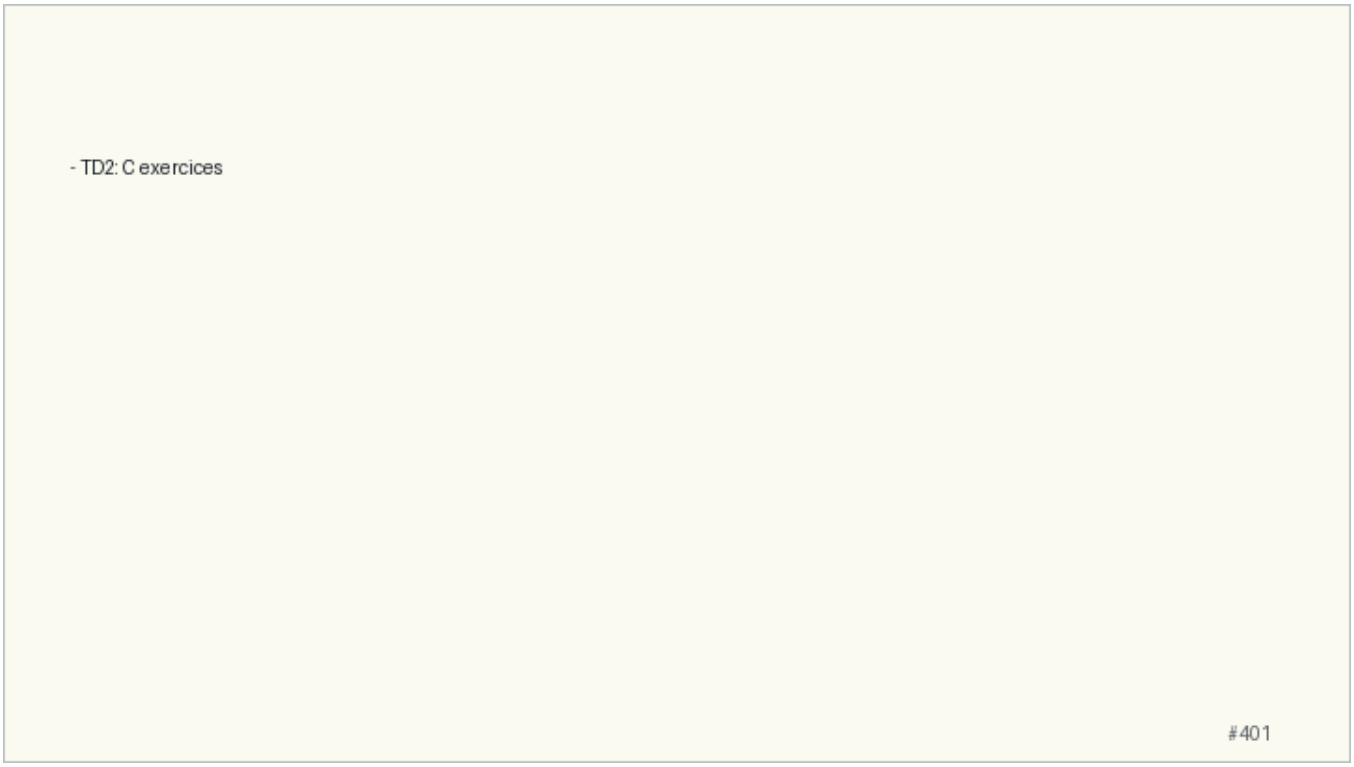
slide 405

slide 406

Anchors: TD2: C exercices; Exercice 1: Basic I/O; Exercice 2: Pointers; Exercice 3: Pointers; Exercice 4: Strings; Exercice 5: Strings and pointers

1.6 Backup: TD2: C exercices

Original slide 401



1.7 Backup: Exercice 1: Basic I/O

Original slide 402

- What prints the following program? (assume array nom is stored at the address 0x3472A000)

```c
#include<stdio.h>
int main(){
char nom[20] = "Balthazar";
double solde = 1000.25;
int age = 21;
printf("Hello %s, you have %d years\n", nom, age);
printf("Your age is written %x in hex\n", age);
printf("Your name start with the letter %c\n", nom[0]);
printf("Your bank account balance is %lf\n", solde);
printf("The address of the table name is %p\n", nom);
}
```

- Exercice 1: Basic I/O

- What prints the following program? (assume array nom is stored at the address 0x3472A000)

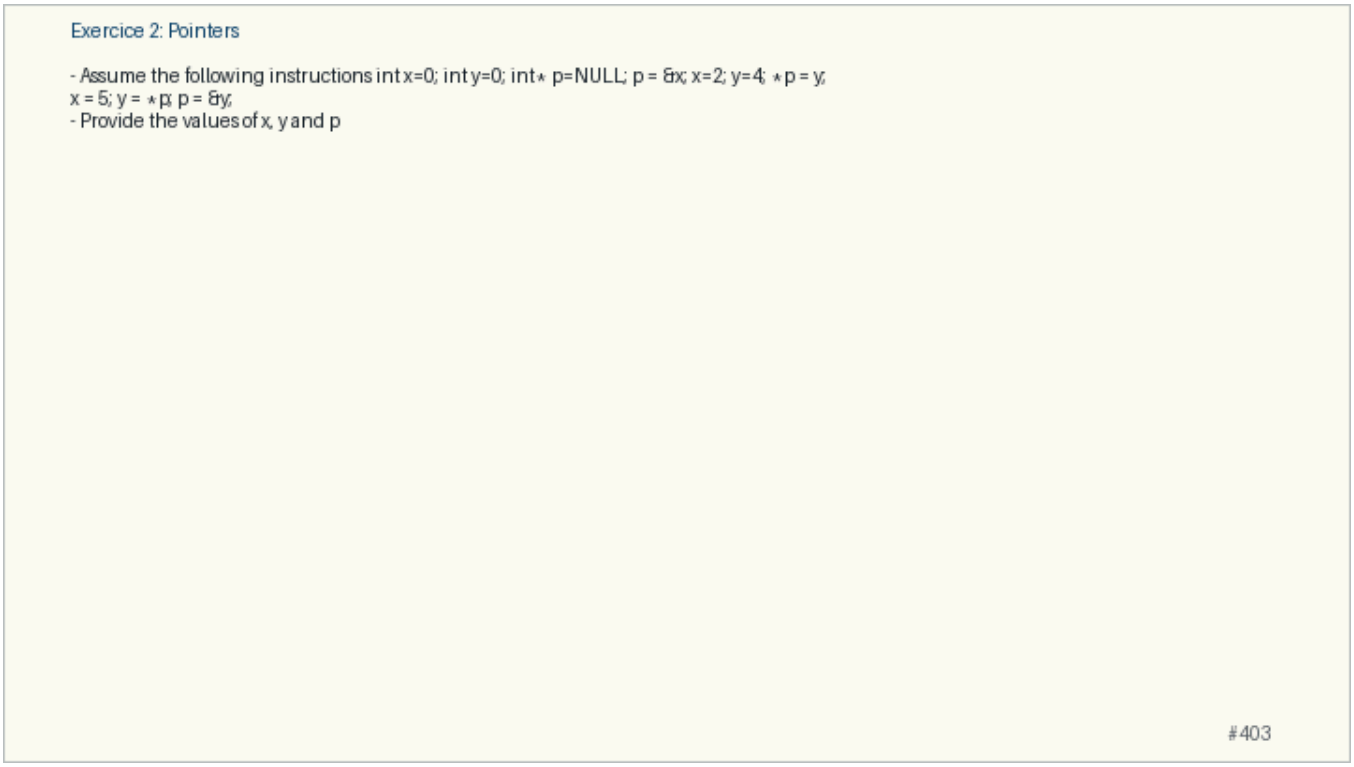

```c
#include<stdio.h>
int main(){
char nom[20] = "Balthazar";
double solde = 1000.25;
int age = 21;
printf("Hello %s, you have %d years\n", nom, age);
printf("Your age is written %x in hex\n", age);
printf("Your name start with the letter %c\n", nom[0]);
printf("Your bank account balance is %lf\n", solde);
printf("The address of the table name is %p\n", nom);
}
```


#402

1.8 Backup: Exercice 2: Pointers

Original slide 403

- Assume the following instructions int x=0; int y=0; int* p=NULL; p = &x; x=2; y=4; *p = y; x = 5; y = *p; p = &y;
- Provide the values of x, y and p



1.9 Backup: Exercice 3: Pointers

Original slide 404

- Give the output of the program

```
#include <stdio.h>
void echange(int * a, int b){
int sauve; sauve = *a; *a = b; b = sauve;
}
void echange2(int * a, int ** b){
int sauve; sauve = *a; *a = **b; **b = sauve;
}
void echange3(int * a,int * b){
int sauve; sauve = *a; *a = *b; *b = sauve;
}
int main(){
int a, b, d; int* c = &d; a = 1; b = 2; (*c) = 3;
echange(&a,b); printf("a= %d, b= %d\n",a,b);
echange2(&a,&c); printf("a= %d, c= %d\n",a,*c);
echange3(&a,&b); printf("a= %d, b= %d\n",a,b);
echange3(&b,c); printf("b= %d, c= %d\n",b,*c);
echange(&a,&b); printf("a= %d, b= %d\n",a,b);
return 0;
}
```

Exercice 3: Pointers

- Give the output of the program

```
#include <stdio.h>
void echange(int * a, int b){
int sauve; sauve = *a; *a = b; b = sauve;
}
void echange2(int * a, int * * b){
int sauve; sauve = *a; *a = * *b; * *b = sauve;
}
void echange3(int * a,int * b){
int sauve; sauve = *a; *a = *b; *b = sauve;
}
int main(){
int a, b, d; int * c = &d; a = 1; b = 2; (*c) = 3;
echange(&a,b); printf("a= %d, b= %d\n",a,b);
echange2(&a,&c); printf("a= %d, c= %d\n",a,*c);
echange3(&a,&b); printf("a= %d, b= %d\n",a,b);
echange3(&b,c); printf("b= %d, c= %d\n",b,*c);
echange(&a,&b); printf("a= %d, b= %d\n",a,b);
return 0;
}
```

#404

1.10 Backup: Exercice 4: Strings

Original slide 405

- Write a function called reverse that reverses the characters of a string, without declaring a second array. The prototype of the function is the following:
- int reverse(char* str) ;

Exercice 4: Strings

- Write a function called reverse that reverses the characters of a string, without declaring a second array. The prototype of the function is the following:
- int reverse(char * str) ;

#405

1.11 Backup: Exercice 5: Strings and pointers

Original slide 406

- Give the output of the following program

```c
#include <stdio.h>
#include <string.h>
char* mess="hello world\n"
int main() {
strcpy(mess+4,mess) ;
return 0;
}
```

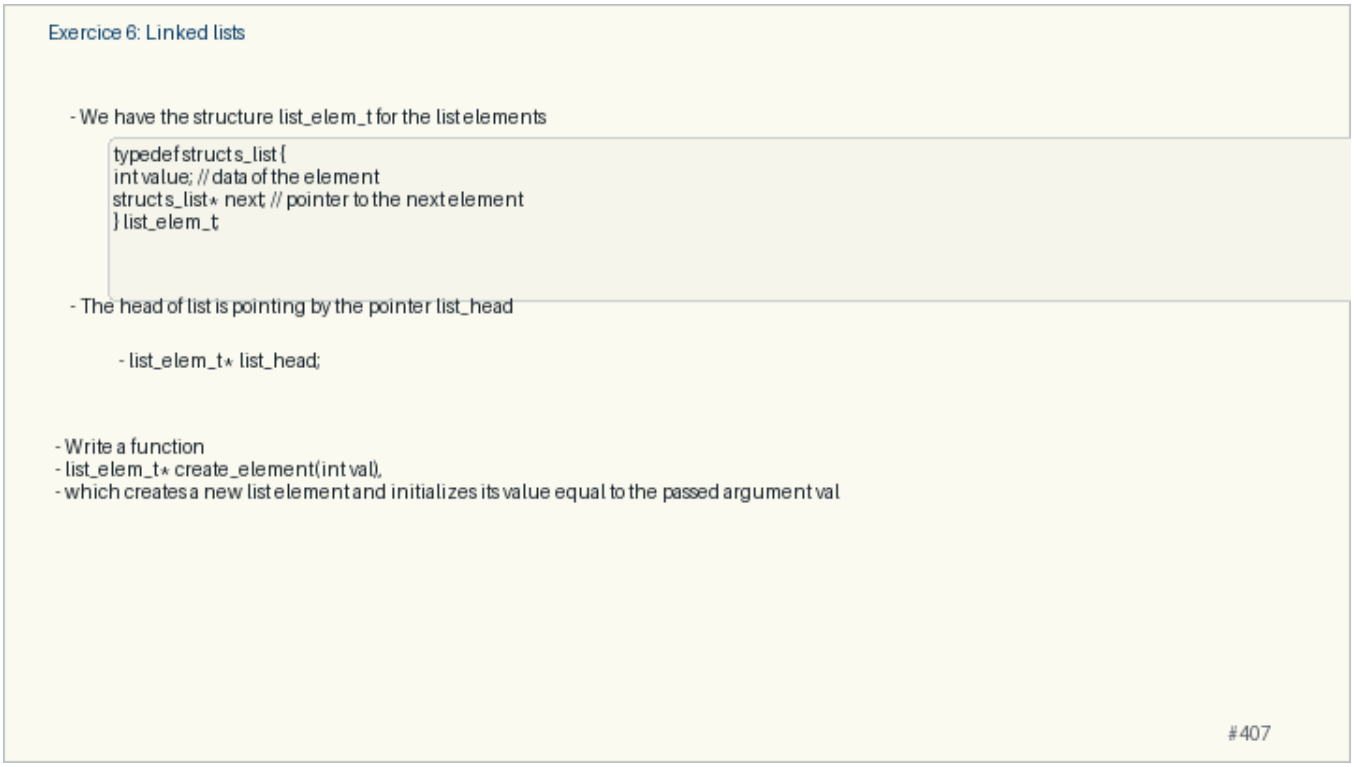
1.12 Backup: Exercice 6: Linked lists

Original slide 407

- We have the structure list_elem_t for the list elements

```c
typedef struct s_list {
int value; // data of the element
struct s_list* next; // pointer to the next element
} list_elem_t;
```

- The head of list is pointing by the pointer list_head
- list_elem_t* list_head;
- Write a function
- list_elem_t* create_element(int val),
- which creates a new list element and initializes its value equal to the passed argument val and the next pointer to NULL



1.13 Backup: Exercice 6: Linked lists

Original slide 408

- Assume the function insert_head, which inserts a new element at the head of the list that has been passed as parameter

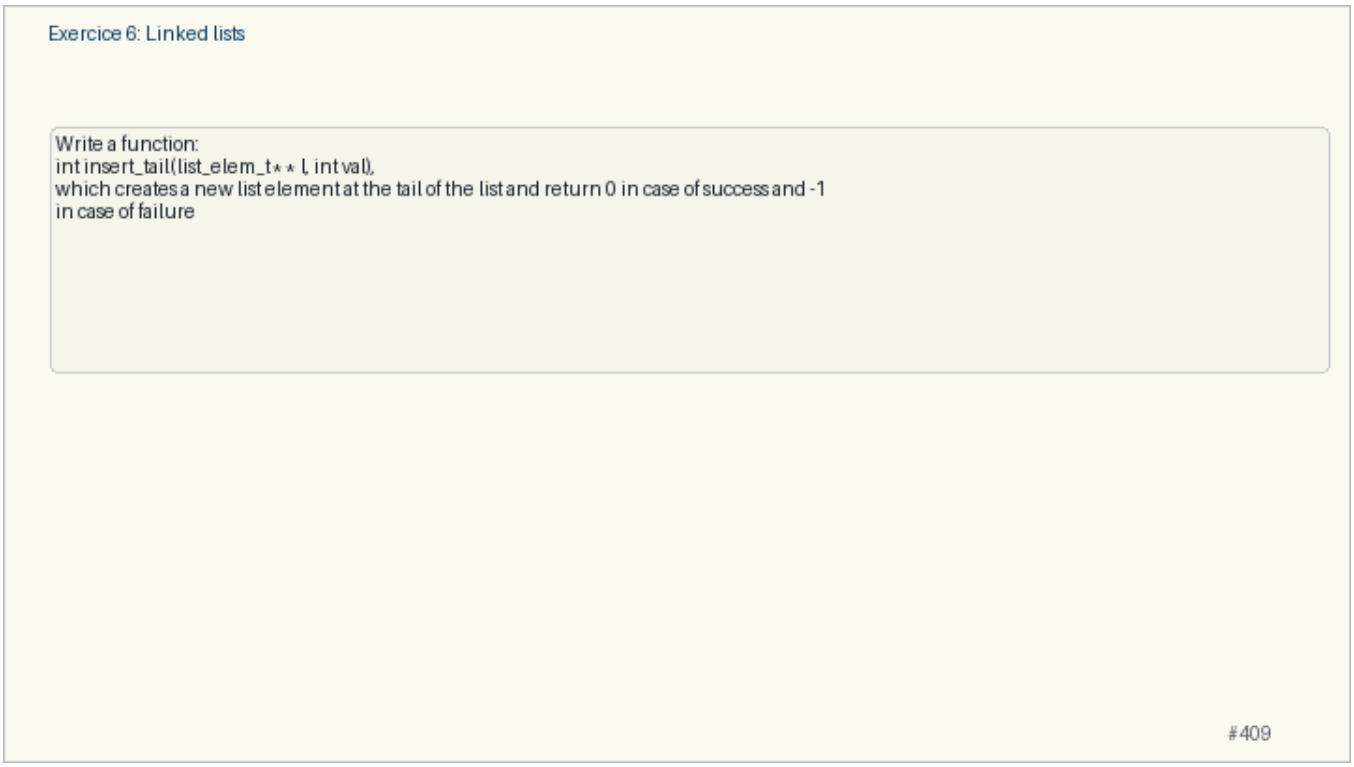
```
int insert_head(list_elem_t* l, int val) {
list_elem_t * nouveau = create_element(val);
nouveau->next = l ;
return 0 ;
}
```

- Is it correct?
- What happens when insert_head(list_head, 1)is executed?
- Correct the function !

1.14 Backup: Exercice 6: Linked lists

Original slide 409

```
Write a function:
int insert_tail(list_elem_t** l, int val),
which creates a new list element at the tail of the list and return 0 in case of success and
-1 in case of failure
```



1.15 Backup: Exercice 7: Files

Original slide 410

- Write a main function that
- Reads the name of a file (name1) from the keyboard
- Reads the name of a file (name2) from the keyboard
- Copies character by character the content of the file name1 to the file name2 and replace all the 'X' with 'Y'.
- If the copy is not possible, prints an error message

```c
#include <stdio.h>
#include <string.h>
FILE* in;
FILE* out;
char nom1[40];
char nom2[40];
int main() {
//To Be Completed
}
```